SMS

BACHELOR OF COMPUTER APPLICATIONS · SEMESTER V

STUDENT MANAGEMENT SYSTEM

A role-based web application for managing students, teachers, classes, attendance, grades and assignments

Project Report
Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Computer Applications (BCA)

Subject: Mobile Application Development
Academic Year: 2026–2027

Project type: Java-based web application with browser-accessible, responsive portals

Submitted by

Student 1: Aniket Dubey Roll No.: 15 Enrollment No.: E24040020100003

Student 2: Mehul Maheshwari Roll No.: 35 Enrollment No.: E24040020100038

Student 3: Surya Pratap Singh Roll No.: 69 Enrollment No.: E24040020100076

Under the guidance of
Mr. Prakash Lambha · Assistant Professor

SRK Institute of Management and Computer Education, Sapeda

Affiliated to K.S.K.V. Kachchh University

ACADEMIC DECLARATION

Certificate

This is to certify that the project entitled **“Student Management System”** has been completed by the students named below in partial fulfillment of the requirements for the Bachelor of Computer Applications (BCA), Semester V, under the subject Mobile Application Development during the academic year 2026–2027.

The report describes the software present in the submitted repository. The implementation is a browser-based Java web application; it is not a native Android application.

Student name	Roll No.	Enrollment No.
1. Aniket Dubey	15	E24040020100003
2. Mehul Maheshwari	35	E24040020100038
3. Surya Pratap Singh	69	E24040020100076

Guide Signature
Name: Mr. Prakash Lambha
Designation: Assistant Professor
Date: _____

Head of Department Signature
Name: _____
Date: _____

Principal Signature
Name: _____
Date: _____

Institute Seal

Date: _____

ACKNOWLEDGEMENT

Acknowledgement

We express our sincere gratitude to our project guide, **Mr. Prakash Lambha, Assistant Professor**, for his guidance, feedback and encouragement throughout the planning and development of this project. His suggestions helped us organize the work into clear modules and consider access control, data integrity and usability alongside implementation.

We thank the Head of the Department and the Principal of **SRK Institute of Management and Computer Education, Sapeda**, affiliated to K.S.K.V. Kachchh University, for providing the academic environment and resources required to complete this work. We also acknowledge the support of our classmates, friends and family members, whose discussions and encouragement helped us refine the system and prepare this report.

Finally, we acknowledge the maintainers of the open-source frameworks and tools used in the project, including Spring Boot, Spring Security, Thymeleaf, Hibernate, Flyway, SQLite, Bootstrap, Maven and Docker.

— The Project Team

PROJECT OVERVIEW

Abstract

Schools often maintain student details, attendance, grades, assignments and notices in separate registers or disconnected files. This makes routine administration slow, increases repeated data entry and makes it difficult for students and teachers to find current information. The Student Management System addresses this problem through a single browser-based application with separate administrator, teacher and student portals. Administrators manage users, classes, subjects, announcements and reports. Teachers work with assigned classes to record attendance and grades, create assignments and review student submissions. Students can view their own academic information, announcements and assignments and submit work online. The application is implemented in Java 21 using Spring Boot, Spring MVC, Spring Security, Thymeleaf, Spring Data JPA, Hibernate, Flyway and SQLite. BCrypt is used for password hashing, and role-based authorization restricts access to portal routes. The project demonstrates how a layered web application can connect a responsive user interface to services and a relational database. It is intended as an academic prototype and a foundation for future improvements such as persistent cloud storage, email verification, PostgreSQL deployment and a native Android client.

Keywords: school management, role-based access control, Spring Boot, SQLite, attendance, assignments, academic reports

REPORT MAP

Table of Contents

Introduction

Objectives of the Project

System Analysis

- Existing and proposed systems
- Feasibility study

System Design

- Architecture and data flow
- Use case, activity, sequence and class diagrams
- Database ER diagram and interface design

System Requirements

Implementation

- Technology stack and modules
- Security and deployment

Testing

Output and Result

Conclusion and Future Enhancements

References

Annexure

CHAPTER 1

Introduction

1.1 Background

Educational institutions manage interconnected information about learners, staff, classes, subjects and academic activity. When these records are stored in separate paper registers or spreadsheets, staff may need to copy the same data several times. Students may also depend on a teacher or office visit to confirm attendance, marks, assignment deadlines or school announcements. A shared information system can make these routine records easier to maintain and access.

Web applications provide a practical way to offer a common system through standard browsers on desktop computers, tablets and phones. A responsive web portal can therefore support mobile access without being a native Android app. The system developed in this repository uses this approach: one Spring Boot application serves role-specific pages to administrators, teachers and students.

1.2 Project purpose

The purpose of the Student Management System is to bring common school administration and classroom workflows into one application. It combines user accounts, role-based dashboards, class and subject configuration, attendance, grading, assignments, submissions, announcements and reports. Data is organized in a relational SQLite database and accessed through Spring Data JPA repositories.

1.3 Scope

- Administrators maintain teachers, students, classes, subjects and announcements, and view school and class reports.
- Teachers access assigned classes, record attendance and marks, publish assignments and announcements, and review submissions.
- Students access their own profile, attendance, grades, announcements and assignments, and submit assignment files.
- The application supports account sign-in, role-based redirection, password hashing, input validation, file-size/type restrictions and generated PDF report cards/ID cards.

1.4 Project boundary

The application is a web-based academic prototype. It does not include a native Android client, email verification, password reset by email, payment processing, school-wide multi-tenancy or an external identity provider. The current configuration uses SQLite and local file storage, so production deployment requires durable database and upload storage planning.

CHAPTER 2

Objectives of the Project

The project objectives were to:

1. Develop a unified portal for key school administration and classroom workflows.
2. Separate administrator, teacher and student capabilities using authenticated roles.
3. Store user accounts and school records in a relational database.
4. Allow teachers to record attendance and marks and manage assignments.
5. Allow students to review their own records and submit assignment work.
6. Provide announcements and summary/reporting features for timely communication.
7. Apply layered application design using controllers, services, repositories and persistence entities.
8. Practice secure password storage, server-side validation, database migrations and containerized deployment.
9. Create a maintainable foundation that can be extended with a native mobile client or production database.

2.1 Success criteria

Area	Success criterion	Evidence in repository
Authentication	Users authenticate and are sent to their role dashboard.	Spring Security configuration and role success handler.
Access control	Routes and records are limited to authorized roles and assigned classes.	Security rules and service/controller checks; authorization tests.
Academic workflows	Attendance, grades, assignments and announcements can be managed through portals.	Teacher, student and admin controllers, services and templates.
Persistence	School records survive application requests and follow a managed schema.	JPA entities/repositories and Flyway migrations.
Reporting	Summary reports and student PDFs can be generated.	Report and PDF service modules.

CHAPTER 3

System Analysis

3.1 Existing system

In a manual or fragmented workflow, a school might use separate registers for attendance, spreadsheets for grades, messaging groups for notices and individual folders for assignments. This can create duplicated records, delayed updates, inconsistent formats and difficulty producing consolidated reports. Access to a record may depend on the availability of the person or document that stores it. These observations describe the common problem domain; they are not a formal audit of a specific institution's current process.

3.2 Proposed system

The proposed system provides one authenticated browser portal backed by shared application services and a relational database. Each role sees functions suited to its work. Administrators configure school data and oversee reports; teachers manage academic activity for their assigned classes; students view their own information and submit work. The system uses server-side authorization and service-level ownership checks to protect requests and data.

Concern	Fragmented approach	Proposed system
Information access	Records spread across registers and files.	Role-specific pages over shared records.
Attendance and grades	Manual lookup and summary calculations.	Teacher forms with stored records and report summaries.
Assignments	Physical collection or separate channels.	Online assignment listing, file submission and review flow.
Announcements	Notices may be distributed through multiple channels.	School-wide and class-targeted portal announcements.
Access control	Physical access or file permissions.	Login, role restrictions and ownership/class checks.

3.3 Feasibility study

Technical feasibility

The project uses mature, open-source Java technologies and a standard browser interface. Java 21 and Maven provide the runtime/build foundation; Spring Boot combines the application components; SQLite minimizes setup for a small prototype. Docker packages the application for cloud hosting. The design is technically feasible for an academic demonstration. A larger school deployment would need stronger operational storage, backups, monitoring and likely a managed database.

Operational feasibility

Users interact with familiar web forms and dashboards. Role separation reduces unrelated options on each portal. Administrators must configure classes, subjects and user accounts before teachers and students can use all workflows. Basic training and clear onboarding would support adoption.

Economic feasibility

The code uses open-source frameworks and can be demonstrated locally without paid software licences. Hosting may be available on a free tier, subject to that provider's current limits. Free container services can sleep when idle and may not provide durable local filesystems; they are suitable for demonstration but should not be assumed to provide production-grade availability or data retention.

Schedule and risk considerations

The project is organized into authentication, administration, teacher, student, reporting and shared-infrastructure modules. Key risks include lost local data on ephemeral hosting, unverified public account details, file-storage durability, and the mismatch between an Android-specific report format and the actual web implementation. These are documented as limitations and future work.

CHAPTER 4

System Design

4.1 Architecture

The application follows a layered MVC design. Thymeleaf renders HTML pages from controller-provided models. Controllers handle HTTP requests and form binding; services enforce workflow rules; Spring Data repositories perform persistence; JPA entities map records to tables. Spring Security authenticates users and applies route rules. Flyway applies schema migrations when the application starts.

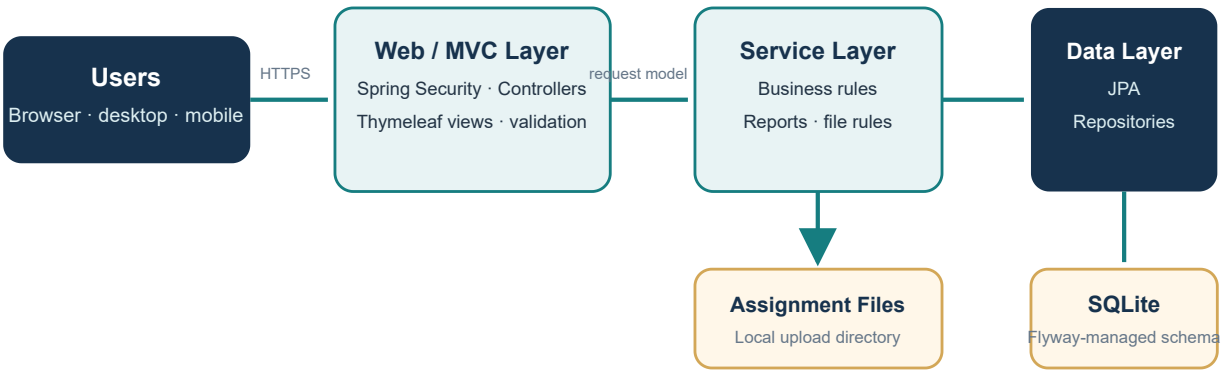


Figure 4.1 — Logical architecture based on the repository's Spring MVC and JPA layers.

4.2 Role use-case diagram

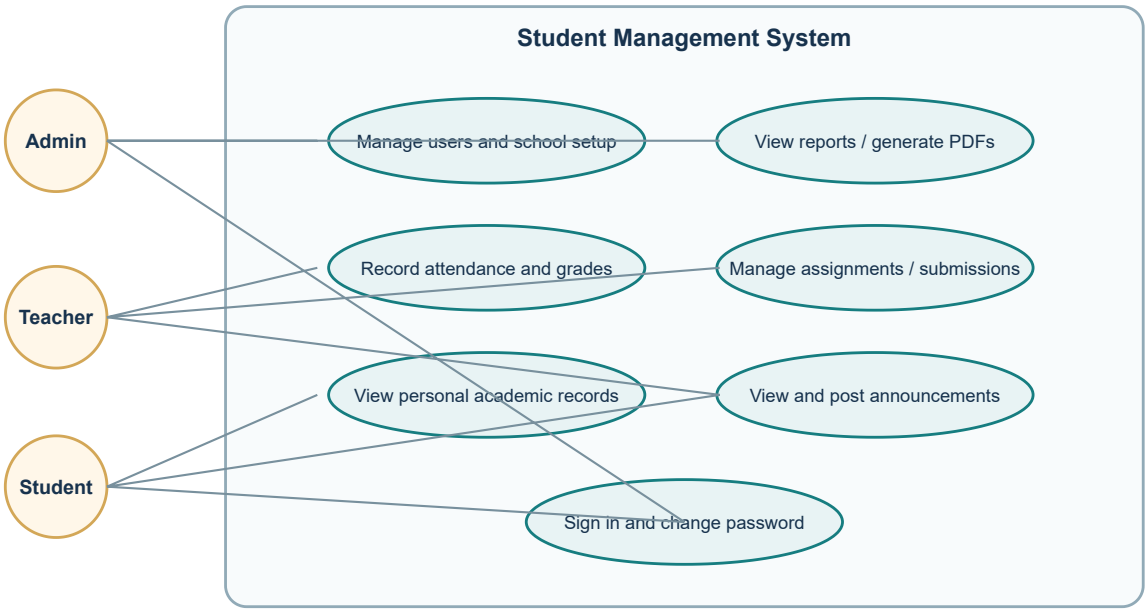


Figure 4.2 — Main use cases grouped by authenticated role. Access is governed by security rules and service ownership checks.

4.3 Data flow diagram

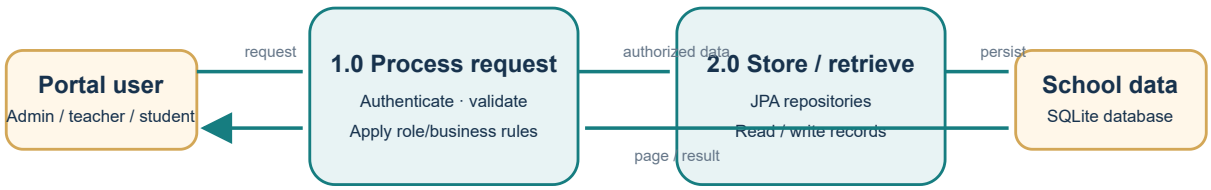


Figure 4.3 — Context-level flow: an authenticated role requests a workflow, rules are applied, and authorized school data is returned.

System Design: UML and Data Model

4.4 Activity diagram — assignment submission

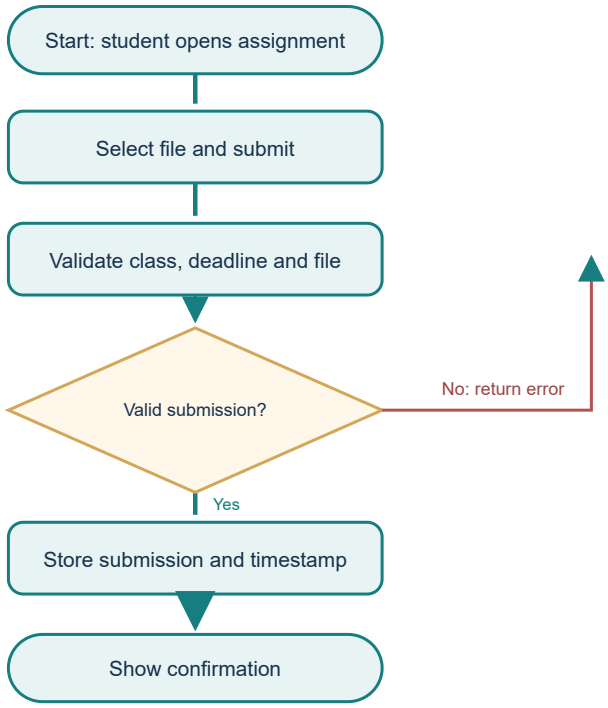


Figure 4.4 — Student assignment submission flow; access and file rules are checked before a submission is stored.

4.5 Sequence diagram — sign-in

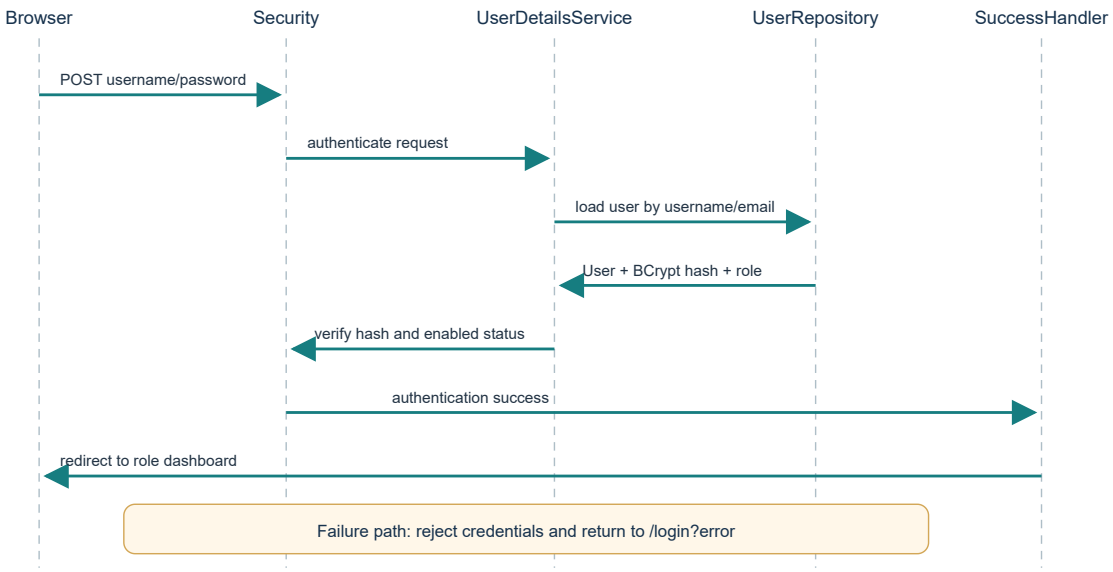


Figure 4.5 — Login sequence from form submission through role-based dashboard redirection.

4.6 Conceptual class diagram

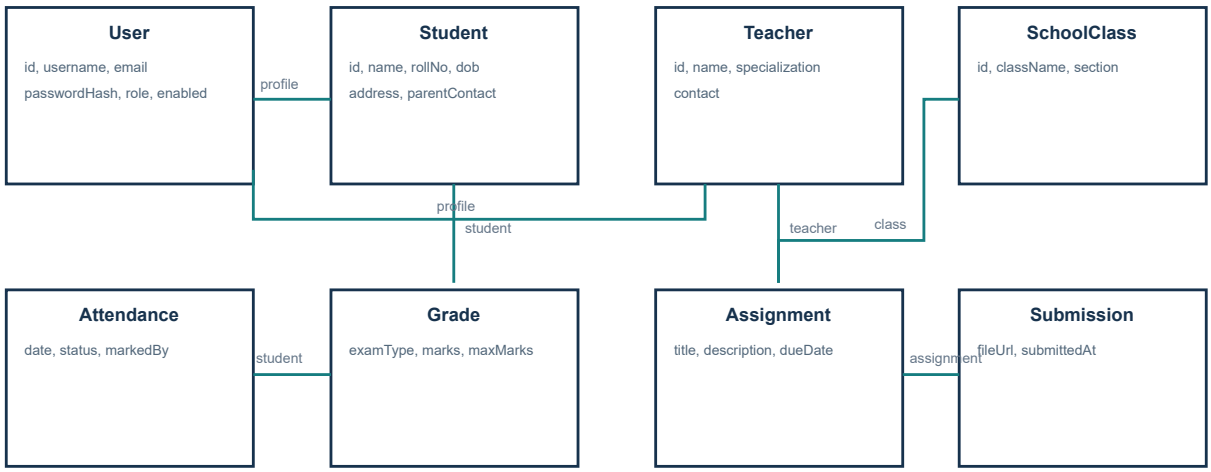


Figure 4.6 — Simplified domain model. User profiles and selected academic relationships are shown; the ER/FK map below lists the complete key relationships.

Database and Interface Design

4.7 ER diagram



Figure 4.7 — Main tables and key relationships. One-to-one user profiles link users to students/teachers; class, subject and academic records are related by foreign keys.

4.8 Interface design

The interface uses a shared visual system with a navy and teal palette, responsive Bootstrap layout, role-specific navigation, cards for dashboard summaries and forms with validation feedback. The following is an illustrative wireframe based on the page structure; it is not a screenshot of a running deployment.

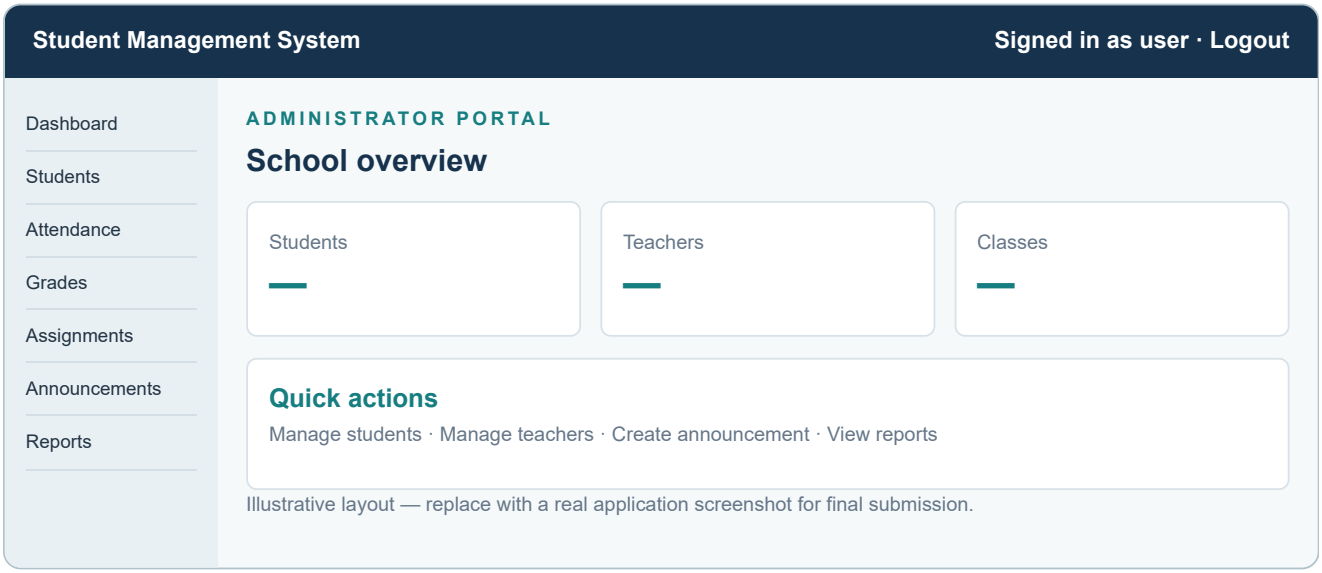


Figure 4.8 — Example dashboard wireframe, not an application screenshot.

4.9 Data integrity rules

- Usernames and email addresses are unique.
- A student roll number is unique within the same class.
- Attendance is unique per student and date.
- A grade is unique per student, subject and exam type.
- A student may have one submission per assignment; resubmission updates the existing record.
- A class/subject pair may have one teaching assignment in the schema.
- Foreign keys connect profiles and academic records to their parent records; Flyway migrations define the schema.

4.10 Key ER relationships

Parent table	Child table / foreign key	Relationship meaning
users	students.user_id; teachers.user_id	Each student or teacher profile is linked one-to-one to a login account.
classes	students.class_id	A class can contain many students; a student may be assigned to one class.
teachers	classes.class_teacher_id	A teacher may be designated as class teacher.
students / teachers	attendance.student_id / marked_by	Attendance belongs to a student and may record the responsible teacher.
students / subjects / teachers	grades.student_id / subject_id / entered_by	Grade records connect a learner, subject and entering teacher.
classes / subjects / teachers	class_subject_teacher foreign keys	TeachingAssignment links a teacher to a subject offered in a class.
teachers / classes / subjects	assignments foreign keys	An assignment is created by a teacher for a class and optional subject.
assignments / students	submissions.assignment_id / student_id	A student submission is connected to an assignment.
users / classes	announcements.posted_by / target_class_id	Announcements have an author and optional class target.

CHAPTER 5

System Requirements

5.1 Hardware requirements

Component	Minimum for demonstration	Recommended
Development computer	Dual-core processor, 4 GB RAM, 2 GB free disk	Modern 4-core processor, 8 GB RAM or more, SSD
Client device	Desktop/laptop/tablet/smartphone with a modern browser	Current Chrome, Edge, Firefox or Safari
Network	Local network for local demo; internet for hosted app	Stable broadband for hosted access and uploads
Server/container	Java 21 runtime, 512 MB RAM is a tight free-tier baseline	At least 1 GB RAM with persistent storage and backups

These are practical demonstration recommendations, not benchmark results or official vendor minimum specifications.

5.2 Software requirements

Software	Project usage	Repository evidence
Java 21	Language/runtime	pom.xml property java.version=21
Spring Boot 4.1.1	Application bootstrap and dependency management	pom.xml parent
Spring MVC + Thymeleaf	Controllers and server-rendered HTML views	Spring web/Thymeleaf dependencies and templates
Spring Security	Form login, password encoder and role route rules	SecurityConfig and auth package
Spring Data JPA / Hibernate	Object-relational persistence	JPA entities and repositories
SQLite + Flyway	Relational database and schema migration	sqlite-jdbc dependency and db/migration scripts
Bootstrap 5.3.3	Responsive page styling	Shared Thymeleaf layout imports CDN stylesheet
Maven 3.9 / Maven Wrapper	Dependency management and build	pom.xml, mvnw and mvnw.cmd
Docker + Eclipse Temurin 21	Container build/runtime for deployment	Dockerfile multi-stage build
Git / GitHub	Source control and hosted deployment source	README deployment workflow

5.3 User requirements

- A supported browser with JavaScript enabled for dynamic interfaces.
- Valid login credentials and network access to the application.
- Administrator-created classes and subjects before complete teacher/student setup.
- For file submission: an allowed file type within the configured 10 MB file limit.

CHAPTER 6

Implementation

6.1 Development process

1. Review the school-management use cases and define role boundaries.
2. Create the schema with Flyway migrations and represent entities with JPA.
3. Implement repositories and services for persistence and workflow rules.
4. Expose portal functions through Spring MVC controllers and Thymeleaf templates.
5. Configure Spring Security, password hashing, role routing and authorization.
6. Add reporting/PDF generation, assignment file handling, demo seeding and Docker packaging.
7. Exercise workflows through automated integration tests and prepare deployment configuration.

6.2 Technology stack

Layer	Technology	Responsibility
Presentation	Thymeleaf, HTML, Bootstrap 5.3.3	Render role-specific responsive forms, tables and dashboards.
Web	Spring MVC, Spring Boot 4.1.1	Route requests, bind forms, validate input and construct views.
Security	Spring Security, BCrypt	Authenticate accounts, hash passwords and guard role routes.
Business	Java services	Enforce ownership, assignment, grading, reporting and validation rules.
Persistence	Spring Data JPA, Hibernate, SQLite	Map and query school data.
Schema	Flyway SQL migrations	Version and apply database structure.
Documents	OpenPDF	Produce report cards and student ID cards.
Deployment	Docker, Java 21 image	Package the application for container hosting.

6.3 Modules

Module	Main responsibilities	Representative package
Authentication	User lookup by username/email, seeded initial administrator, role redirect and account management.	com.sms.auth
Administration	Manage students, teachers, classes, subjects, announcements and reports.	com.sms.admin
Teacher portal	Assigned classes, attendance, grades, assignments, submissions, announcements and reports.	com.sms.teacher
Student portal	Student dashboard, profile/password, attendance, grades, assignments and announcements.	com.sms.student
Reporting	Attendance/performance summaries, report cards and ID card PDFs.	com.sms.report
Announcements	School-wide/class-targeted notices and role-aware visibility.	com.sms.announcement
Shared infrastructure	Security configuration, file storage, common errors and converters.	com.sms.common

6.4 Role-specific functionality

Administrator

Creates and updates student and teacher accounts, configures class/section and subject records, links teachers to class-subject assignments, publishes school/class announcements and reviews reports. The initial administrator is created only when none exists and an admin password is configured.

Teacher

Works with assigned classes and subjects, records attendance and grades, creates assignments with deadlines, reviews student submissions, publishes permitted announcements and views teacher-scoped reports. Service logic limits actions to the teacher's assigned classes and subjects.

Student

Views own profile, attendance, marks, assignment details and announcements. A student can upload and resubmit assignment work subject to class membership, deadline and file validation. Students cannot access teacher or administrator portal routes.

Public registration

The sign-up form offers Student, Teacher and Administrator choices and dynamically displays role-specific fields. Student registration collects class and roll number; teacher registration collects specialization/contact; administrator registration uses an invitation code. Separate server-side invitation codes are configured as `SMS_TEACHER_SIGNUP_CODE` and `SMS_ADMIN_SIGNUP_CODE`. These codes prevent unauthenticated visitors from freely granting themselves staff or administrator privileges. The current student form does not verify the submitted identity against an external school roster.

6.5 Security and validation

- Form-based sign-in accepts username or email; user passwords are stored as BCrypt hashes.
- Role rules restrict `/admin/**`, `/teacher/**` and `/student/**` routes.
- Service checks enforce ownership and class/subject boundaries for teacher and student actions.
- Bean Validation and business checks protect form input; uniqueness rules are enforced in service/database logic.
- Uploaded files are restricted by extension and size and stored under generated names.
- CSRF protection is provided by Spring Security's default form configuration.

6.6 Deployment configuration

The Dockerfile uses a Maven build stage and a Java 21 runtime stage. Maven is configured to use a Google-hosted mirror of Maven Central for artifact resolution, with bounded retries for HTTP 429 rate limits. The runtime listens on the hosting platform's `PORT`. Render can build the Dockerfile from the repository. The current SQLite file and upload directory are local paths; they need persistent volumes or replacement managed services before production use.

CHAPTER 7

Testing

7.1 Testing approach

The repository contains JUnit/Spring test classes covering application routing, role access, administrator workflows, student privacy, teacher workflows, announcements, reports, PDF generation and assignment submission. These tests are intended to exercise integrated web, service and persistence behavior. This report records the tests found in source; it does not claim they were executed during PDF preparation.

7.2 Test types represented

- **Integration tests:** Spring application context, MVC routes, database-backed services and page rendering.
- **Functional tests:** create/update, attendance, grading, assignment and announcement workflows.
- **Authorization tests:** role route restrictions, teacher ownership and student-specific data visibility.
- **Output tests:** PDF report-card/ID-card output and expected data content.
- **Validation tests:** duplicates, invalid dates/marks, deadlines, file rules and password changes.

7.3 Representative test-case matrix

ID	Description	Expected result	Repository test evidence	Execution status for this report
TC-01	Open portal without authentication.	Redirect to sign-in.	SmsApplicationTests.anonymousUserIsSentToLogin	Not rerun for report.
TC-02	Student requests administrator dashboard.	Access denied.	SmsApplicationTests.studentCannotOpenAdminDashboard	Not rerun for report.
TC-03	Teacher records attendance for assigned class.	Records are saved/updated without duplicates.	TeacherPortalTest.attendanceIsSavedThenUpdatedNotDuplicated	Not rerun for report.
TC-04	Teacher enters a mark greater than maximum.	Submission is rejected and no invalid grade is saved.	TeacherPortalTest.marksAboveMaxRejectWholeSubmission	Not rerun for report.
TC-05	Student views attendance and grades.	Only the signed-in student's records are displayed.	StudentPortalTest.attendanceShowsOnlyOwnRecords; gradesShowOnlyOwnMarks	Not rerun for report.
TC-06	Teacher announces to a class outside their assignment.	Request is rejected; no announcement is created.	AnnouncementTest.teacherCanOnlyPostToOwnClasses	Not rerun for report.
TC-07	Student submits and resubmits assignment file.	One submission is retained with updated file metadata.	AssignmentFlowTest.studentSubmitsResubmitsAndTeacherDownloads	Not rerun for report.
TC-08	Generate report card and student ID PDF.	Valid PDF bytes contain the correct student/report data.	ReportTest.pdfsAreRealAndContainTheStudentsData	Not rerun for report.
TC-09	Use a duplicate username or short password in admin form.	Form returns field validation feedback.	AdminManagementTest.duplicateUsernameAndShortPasswordAreFieldErrors	Not rerun for report.

> **How to report final results:** Run `mvnw.cmd test` (Windows) or `./mvnw test` (Linux/macOS), record the actual pass/fail totals, and update the status column before submission. Do not describe tests as passed unless the run

> completes successfully.

7.4 User acceptance testing

A formal user acceptance session with school administrators, teachers or students is not documented in the repository. Before academic demonstration, perform a guided walkthrough with sample records and capture reviewer feedback. The checklist in the annexure provides a repeatable demonstration path.

CHAPTER 8

Output and Result

8.1 Result summary

The source code implements a role-based school management web application with separate portal workflows and a shared data model. Its major outcomes are centralized management of users and school structure, teacher tools for attendance/grades/assignments, student access to personal academic records, announcements, and administrative reporting. Generated report cards and ID cards provide printable outputs.

Objective	Implemented capability	Evidence location
Role-based portal	Admin, teacher and student dashboard routing and protected paths.	auth, common security, role-specific controllers/templates
Centralized school records	Students, teachers, classes, subjects and assignments stored in the database.	school entities, repositories, Flyway migrations
Academic workflows	Attendance, grades, assignments and submissions.	teacher and student modules
Communication	School-wide and class announcements.	announcement module and templates
Reports	Attendance/performance summaries, report cards and ID cards.	report module and PDF service

8.2 Application screenshots

No authentic application screenshots were present in the repository files available for this report. The wireframe in Chapter 4 is an illustration only, not a captured application output. Insert screenshots from the deployed or locally running application in the frames below before final submission; remove any real student personal data or credentials first.

Screenshot 1 — Sign-in / sign-up

Capture the login page and role selector after deployment.

[Insert real screenshot here]

Screenshot 2 — Administrator dashboard

Show overview cards and administrator navigation.

[Insert real screenshot here]

Screenshot 3 — Teacher attendance or grade entry

Use a class with sample data only.

[Insert real screenshot here]

Screenshot 4 — Student portal / assignment submission

Show student-facing records or submission confirmation.

[Insert real screenshot here]

8.3 Working demo explanation

1. Open the sign-in page and explain how the authenticated user is redirected according to role.
2. Sign in as an administrator and create a class, subject, teacher and student.
3. Assign a teacher to the class/subject, then demonstrate attendance and grade entry.
4. Publish an announcement and create an assignment with a due date.
5. Sign in as the student, verify personal attendance/grades, view the assignment and upload a permitted sample file.
6. Return as administrator and generate a class/school report, report card or ID card.
7. Explain route restrictions, password hashing and current storage limitations.

CHAPTER 9

Conclusion and Future Enhancements

9.1 Conclusion

The Student Management System demonstrates how a layered web application can organize common school workflows under a shared sign-in and data model. Its administrator, teacher and student portals provide different capabilities while reusing business services and database records. The project brings together Java programming, MVC web development, persistence, schema migration, authentication, access control, validation, file handling, PDF generation and container packaging.

The work also illustrates the importance of accurately matching a project deliverable to its academic format. The supplied template is for Android development, whereas this repository contains a responsive web application. The project should be presented as a web system unless a separate Android client is implemented and approved by the guide.

9.2 Learning outcomes

- Organizing an application into controller, service, repository and entity layers.
- Designing relational entities, keys, constraints and incremental schema migrations.
- Implementing authentication, BCrypt password hashing and role-aware route security.
- Building and validating data-entry workflows with Spring MVC and Thymeleaf.
- Applying authorization rules to class ownership and student-specific records.
- Generating documents and handling validated assignment uploads.
- Containerizing an application and identifying the operational risks of ephemeral storage.
- Documenting functionality and tests without claiming unverified results.

9.3 Future enhancements

1. **Persistent production storage:** migrate SQLite to PostgreSQL and assignment files to durable object storage; add automated backups and recovery procedures.
2. **Native Android client:** design a mobile API, token-based authentication and Android screens if a native mobile deliverable is required.
3. **Account verification:** verify student identity against an administrator-maintained roster; add email verification, password reset and account approval.
4. **Improved staff onboarding:** issue time-limited invitations or require administrator approval for teacher and administrator accounts.
5. **Operational security:** configure HTTPS, rate limiting, audit logs, secret rotation, security headers and monitoring.
6. **Accessibility and UX:** complete keyboard/screen-reader checks, improve mobile data-entry flows and localize date/number formats.
7. **Additional school functions:** timetable, fees, parent/guardian portal, messaging, academic calendar and exportable spreadsheets.
8. **Quality assurance:** automate the complete test suite in CI, add browser-level acceptance tests and conduct usability evaluation.

CHAPTER 10

References / Bibliography

Repository-specific descriptions in this report are based on the source code, README, Maven configuration, database migration files and automated test source. The following official documentation is relevant to the technologies used:

1. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/reference/
2. Spring Security Reference. https://docs.spring.io/spring-security/reference/
3. Spring Framework Reference Documentation. https://docs.spring.io/spring-framework/reference/
4. Thymeleaf Documentation. https://www.thymeleaf.org/documentation.html
5. Hibernate ORM User Guide. https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html
6. Flyway Documentation. https://documentation.red-gate.com/flyway
7. SQLite Documentation. https://www.sqlite.org/docs.html
8. Bootstrap Documentation. https://getbootstrap.com/docs/5.3/
9. Apache Maven Documentation. https://maven.apache.org/guides/
10. Docker Documentation. https://docs.docker.com/
11. Render Documentation — Docker Deploys. https://render.com/docs/docker
12. SRK Institute of Management and Computer Education, Sapeda; K.S.K.V. Kachchh University — academic project report format supplied for BCA Semester V, 2026–2027.

ANNEXURE

Project Walkthrough and Submission Checklist

A. Repository map

Path	Purpose
src/main/java/com/sms/auth	Authentication, role handling, account service and user persistence.
src/main/java/com/sms/admin	Administrator controllers, forms and school management services.
src/main/java/com/sms/teacher	Teacher portal and teaching workflows.
src/main/java/com/sms/student	Student portal and student workflows.
src/main/java/com/sms/school	JPA entities and repositories for core school records.
src/main/java/com/sms/report	Report calculations and PDF services.
src/main/resources/templates	Thymeleaf HTML views for each portal.
src/main/resources/db/migration	Flyway database migrations.
src/test/java/com/sms	Spring and JUnit tests for access and workflows.
Dockerfile / pom.xml	Container build and Maven dependencies/configuration.

B. Suggested presentation script

1. **Problem:** explain the cost of scattered academic records and delays.
2. **Solution:** show a single web system with role-specific portals.
3. **Design:** walk through the architecture and ER diagrams.
4. **Security:** show login, hashed passwords, role route rules and data ownership.
5. **Demonstration:** follow the admin → teacher → student workflow from Chapter 8.
6. **Testing:** show a test report generated by running the test suite; discuss authorization and validation cases.
7. **Limitations:** explain SQLite/local uploads and the Android-template/web-project difference clearly.
8. **Future scope:** propose PostgreSQL, durable uploads, verification and an optional Android app.

C. Items to complete before printing/submission

- Enter each student's full name, roll number and enrollment number on the cover and certificate.
- Confirm the project category with the guide because the supplied format expects Android while this repository is web-based.
- Run the Maven test suite and update the results with actual pass/fail output.
- Capture genuine screenshots from the deployed or local app; replace the four screenshot placeholders.
- Confirm guide/HOD/Principal names, dates, college letterhead and signature/seal requirements.
- Review deployment configuration and use only synthetic/demo data in screenshots.
- Print the final PDF on A4 paper and check page breaks, signature space and screenshots before binding.

> **Demonstration credentials:** Never include active passwords, invitation codes, database credentials, real student records or other secrets in the report or screenshots. Use synthetic accounts and reset any credentials exposed during development.